REPORT ASSIGNMENT 2

Introduction

The project implements a 2D free-surface fluid solver based on semi-discrete optimal transport. The project was developed progressively through Labs 6, 7, and 8. First, Voronoi cells are constructed. Next, weights are assigned to the sites to obtain a power diagram, and these weights are then optimized using L-BFGS. Finally, these cells are used to move fluid particles.

The idea is that each particle has a cell surrounding it. By controlling the area of these cells, we can represent an incompressible fluid, since each particle retains approximately the same volume of fluid.

Validation of basic geometric functions

Before discussing the simulation, here are the tests for the basic geometric functions that have been implemented and are used on polygons.
To test these functions, I manually created a unit square with vertices (0,0), (1,0), (1,1), and (0,1), then calculated its area, centroid, and the integral of the squared distance to the center.

```
Polygon square;
square.vertices.push_back(Vector(0.0, 0.0));
square.vertices.push_back(Vector(1.0, 0.0));
square.vertices.push_back(Vector(1.0, 1.0));
square.vertices.push_back(Vector(0.0, 1.0));

std::cout << "Square area = " << square.area() << std::endl;
Vector c = square.centroid();
std::cout << "Square centroid = (" << c[0] << ", " << c[1] << ")" << std::endl;
std::cout << "Integral distance to center = " << square.integral_square_distance(Vector(0.5, 0.5)) << std::endl;
```

The program produces the following results:
Square area = 1
Square centroid = (0.5, 0.5)
Integral distance to center = 0.0833333

This test verifies that the polygon representation is consistent and that the basic functions work correctly on a simple case for which the result is known.

LAB 6 - Voronoi diagram

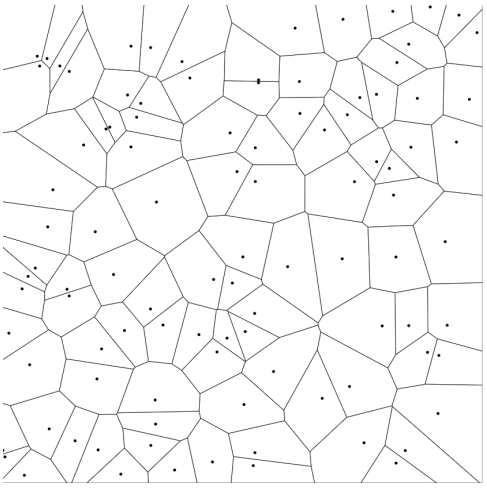
In Lab 6, we implement a 2D Voronoi diagram. The simulation domain is the unit square.

For each point, we want to find the region of the square that is closer to that point than to all other points. To construct a cell, we start with the entire square. Then, I clip this polygon with the bisectors between the current point and all other points. With each cut, I keep only the part of the polygon that is closer to the current point. After comparing with all other points, the remaining polygon corresponds to the Voronoi cell.

The polygons are clipped using the Sutherland-Hodgman algorithm. Since the cells are independent, the loop over the cells is parallelized using OpenMP.

To verify that the construction of the Voronoi diagram works, I generated a test with 100 random sites in the unit square. The result is a set of polygonal cells that fill the domain.

The first image shows the Voronoi diagram with the points displayed in black. We can see that each cell is associated with one site and that the boundaries between cells separate the regions associated with the different points. This demonstrates that the successive clipping is working.



To verify the result numerically, I also calculated the sum of the areas of all the cells. The program returns: total area = 1

This shows that the resulting cells do indeed cover the entire domain.
This confirms that the construction of the Voronoi diagram is consistent for this test.

LAB 7 : Power diagram and weights optimization

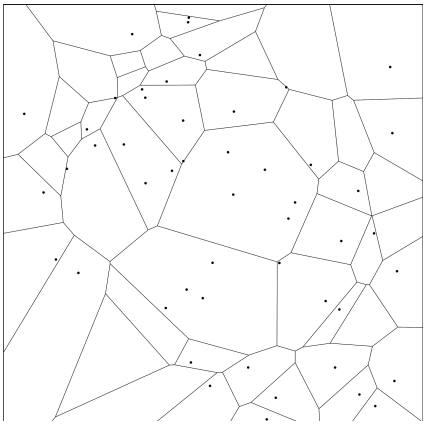
In Lab 7, we extend the Voronoi diagram to a power diagram.
In a classic Voronoi diagram, we only compare the distances between a point on the plane and the sites.
In a power diagram, we assign a weight to each site and compare the weighted distances: $||x - Pi||^2 - wi$

The weights allow us to adjust the size of the cells. A cell can become larger or smaller depending on the value of its weight.

Next, we use L-BFGS to optimize the weights. The goal is for each cell to have a specified area. At each iteration, I construct the power diagram, calculate the area of each cell, and then compare that area with the target area. The gradient corresponds to the difference between the current area and the desired area. L-BFGS modifies the weights to obtain cells with areas close to the target.

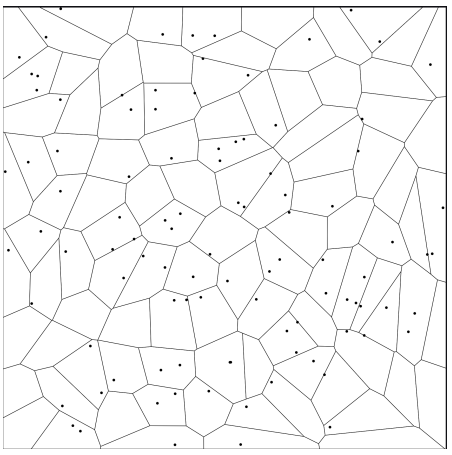
This step corresponds to semi-discrete optimal transport: we transport a uniform continuous density to a set of discrete particles, by assigning a mass to each cell.

To verify the extension of the Voronoi diagram into a power diagram, I generated 50 random points in the unit square with non-zero weights. I chose larger weights for points near the center to make the effect of the weights visible.



We can see that the cells are no longer determined solely by their distance from the sites. The weights shift the boundaries between the cells, which changes their shape and area. In particular, we observe that some cells are larger than in a classic Voronoi diagram.

To test the weight optimization, I generated N random sites within the unit square, whose area is 1. Thus, with N sites, the target area of a cell is 1/N.
At each iteration, the program reconstructs the power diagram, calculates the area of each cell, and then compares this area with the target area. L-BFGS uses this error to adjust the weights.



After optimization, the resulting cells have areas closer to the target value. This test therefore demonstrates that the weight optimization works.

Lab 8 : Free-surface fluid simulation

In Lab 8, we use optimized Laguerre cells to simulate a fluid. The fluid particles are initialized within a disk located inside the unit square. Each particle has a position and a velocity.

To create a free surface, the fluid must not fill the entire square. We therefore add an additional variable representing air. If the fluid occupies a fraction (f) of the domain, then

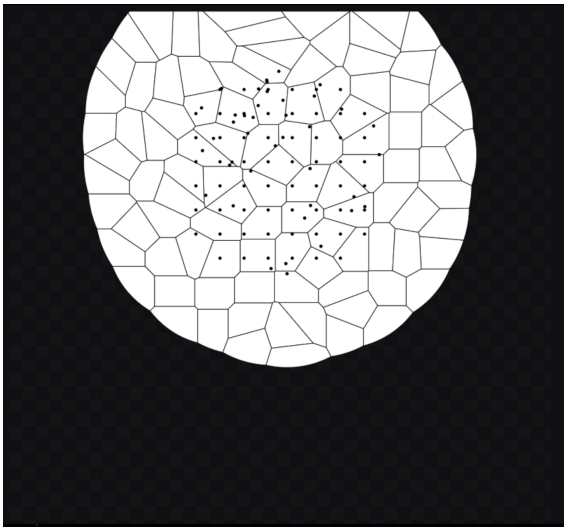
each fluid cell must have an area of (f/N), and the remainder of the domain corresponds to air.

Each fluid cell is also intersected by a disk centered on the particle. The radius depends on the difference between the particle weight and the air weight. Since the clipping algorithm works with polygons, the disk is approximated by a polygon with many sides.

At each step of the simulation, I compute the Laguerre cells, optimize the weights using L-BFGS, calculate the centroid of each cell, and apply a spring force that pulls each particle toward the centroid of its cell. I add gravity and update the velocities and positions.

The particles are confined within the unit square using simple boundary conditions. If a particle touches a boundary, its position is corrected and its velocity is reversed with damping.

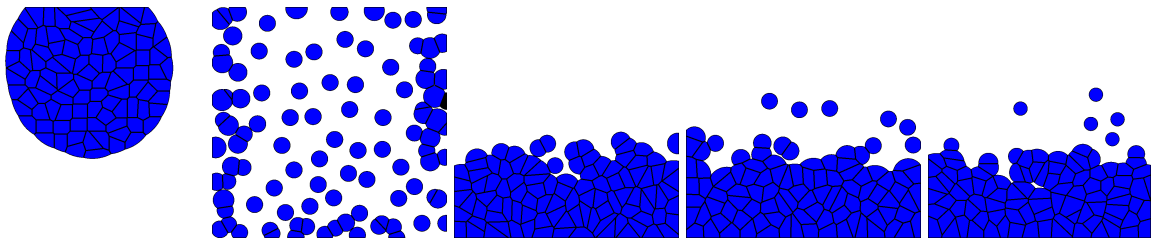
To verify that the initial fluid setup works, I ran a test with 100 particles. The particles are first initialized in a compact region, then I apply L-BFGS optimization to compute the initial Laguerre cells.



The resulting image shows that the cells are clustered in a compact region and bounded by a free boundary.

The total area of the fluid cells matches the specified volume fraction:
The program reports Initial total fluid area = 0.400184, Target fluid volume = 0.4.

To test the full simulation, I ran 1,000 steps with 100 particles and a time step of dt = 0.002. I saved several images at times 0, 100, 300, 600, and 999.



The images show that the fluid cells move downward under the influence of gravity, and the fluid mass gradually accumulates at the bottom of the domain. This demonstrates that the fluid simulation is working correctly.

The full video of the simulation is also available in the GitHub repository under the name `output.mp4`. It was generated from the frames saved during the simulation.

Course feedbacks

I found the course really interesting. I plan to pursue a master's degree in computer graphics, so it was very enriching for me.
In my opinion, the textbook is very well written and provides excellent guidance during the lab sessions. However, I sometimes found the lab instructions to be vague, and I had trouble understanding what I was supposed to do. Since we had to do everything in class, I think it would have been better to provide more guidance for the labs.